Study: bảng tổng hợp tài liệu

RL cho VLA tổng quát; diffusion và flow matching trong sinh hành động

1. Phương pháp RL cho VLA và policy flow

Bảng 1: Ma trận so sánh các bài RL cho VLA và policy flow.

Bài	Model nền, dạng action	Mục tiêu học	Reward	Cách huấn luyện, phần được update	Giả định	Kết quả chính	Câu hỏi còn mở
SimpleVLA-RL	OpenVLA-OFT 7B, head sửa thành token; **token**	GRPO, không critic, bỏ KL; clip [0,8; 1,28]; bỏ nhóm toàn thắng/toàn thua; nhiệt độ rollout 1,6	nhị phân 0/1 theo kết quả, gán cho mọi token của trajectory	online, sim; SFT trước rồi RL; 8×A800; update toàn bộ	SFT phải có success > 0; reset được môi trường	LIBERO TB 91,0→99,1, SFT đầy đủ; SFT 1 traj./task: 48,9→96,9, Long 17,3→91,7	học khi success ≈ 0; policy học đường tắt khác demo, đẩy vật thay vì gắp, "pushcut"
$\pi_{\mathbf{RL}}$	π_0, $\pi_{0.5}$, 3,3B; **flow**	PPO + GAE ở mức chunk; log-likelihood qua Flow-Noise hoặc Flow-SDE	nhị phân; ManiSkill thêm 0,1 khi gripper chạm vật	online, sim, khung RLinf; SFT few-shot trước; chỉ update action expert ∼300M, VLM đóng băng	có SFT trước; sim đủ nhanh để rollout online	LIBERO few-shot π_0: 57,6→97,6 (Flow-Noise) / 96,1 (Flow-SDE); $\pi_{0.5}$: 77,1→98,3 / 97,9	cần nhiều rollout và reset; Flow-Noise hay Flow-SDE tốt hơn tuỳ model; MetaWorld OOD không cải thiện
$\pi^*_{0.6}$ / **RECAP**	$\pi_{0.6}$: Gemma 3 4B + action expert 860M; **flow** (+ token FAST phụ)	advantage conditioning: $-\log\pi(a \mid o,\ell) - \alpha\log\pi(a \mid I,o,\ell)$, $I = \mathbb{1}(A > \epsilon_\ell)$; value phân phối 201 bin, VLM 670M; không cần log-prob chính xác	-1 mỗi bước, 0 nếu thành công, $-C_{\text{fail}}$ nếu thất bại	lặp offline trên robot thật: thu rollout + can thiệp của người, huấn luyện lại từ checkpoint pretrain; update toàn bộ VLA + value function	quan sát Markov; can thiệp của người luôn tốt (I = True); có người gán reward, can thiệp, reset	robot thật: throughput tăng hơn 2 lần, tỉ lệ thất bại giảm khoảng một nửa ở task khó; success > 90% trừ task giặt đồ đa dạng	phụ thuộc người reset, gán reward, can thiệp
RIPT-VLA	QueST ∼20M (token); OpenVLA-OFT 7B (hồi quy liên tục); **token / liên tục**	PPO clip + advantage leave-one-out (RLOO), $K = 16$ (QueST) / 8 (OFT); bỏ context toàn thắng/toàn thua	nhị phân 0/1, không critic	online, sim; OFT cần thêm head dự đoán scale Laplace để có log-prob; OFT: LoRA $r = 32$ + action head	SFT có success > 0; reset được đúng context ban đầu cho cả nhóm rollout	LIBERO, QueST: 82,7→93,6, Long 68,8→87,5; OFT: 96,7→97,5; 1 demo: 3,5%→97,2%	học khi success ≈ 0; SFT đã mạnh thì RL tăng ít (OFT +0,8)
VLA-RL	OpenVLA-7B; **token**	PPO + GAE, có critic, critic warmup; log-prob = tổng log-prob của 7 token action	sparse của môi trường + reward model (RPRM) quá trình học từ nhãn giả : tách bước con theo độ mở gripper và vận tốc ≈ 0	online, sim (LIBERO 4 suite, mỗi suite train riêng); LoRA	nhãn giả từ gripper/vận tốc khớp ranh giới bước con	LIBERO TB 76,5→81,0, Long 53,7→59,8	RPRM dựa trên heuristic, tác giả thừa nhận khó áp dụng cho thao tác khéo léo
FPO	không dùng model pretrain: policy flow train từ đầu (MuJoCo Playground, humanoid); **flow**	PPO-clip, thay tỉ lệ likelihood bằng $\exp(\hat{\mathcal{L}}_{\text{CFM},\theta_{\text{old}}} - \hat{\mathcal{L}}_{\text{CFM},\theta})$; $\epsilon_{\text{clip}} = 0{,}05$; không cần log-prob chính xác	reward của môi trường (DeepMind Control; humanoid theo DeepMimic)	online, sim; update policy + value function như PPO	loss flow matching bằng $-$ELBO cộng hằng số, hằng số triệt tiêu trong tỉ lệ	MuJoCo TB 759,3 so với PPO Gauss 667,8; humanoid chỉ theo root: 54,3% so với 29,8%	chưa thử trên policy pretrain bằng BC; thiếu công cụ như ước lượng KL
π_0**-FPO**	π_0 **đóng băng**; một actor flow nhỏ sinh latent, π_0 giải mã latent thành action; **flow**	FPO: chênh lệch loss flow matching, chuẩn hoá theo batch, $\rho = \exp(\beta z)$, rồi PPO-clip; critic ensemble + GAE	reward thưa của môi trường, không mô tả chi tiết	online, sim (LIBERO 4 suite, ALOHA); chỉ update actor latent + critic	giảm loss flow matching tương ứng tăng mật độ, chưa chứng minh	LIBERO TB 87,2, Long 65,3 so với π_0-FAST 85,5 / 60,2	không báo seed, phương sai, siêu tham số; không sửa trọng số π_0 nên gần DSRL hơn là fine-tune VLA
DSRL	policy diffusion/flow bất kỳ, kể cả π_0 3,3B; **flow / diffusion**	học policy chọn nhiễu đầu vào w, coi base policy là một phần môi trường; SAC hoặc DSRL-NA (2 critic)	reward của môi trường; robot thật dùng 0/1	online, offline hoặc offline-to-online; sim và robot thật; base policy **đóng băng**, chỉ cần chạy forward	base policy xử lý được nhiễu khác nhau cho hành vi khác nhau	π_0 trên LIBERO: ≈20%→≈100% sau ≈10 000 mẫu; robot thật, π_0 bật máy nướng bánh: 5/20→18/20	bị giới hạn bởi hành vi base policy làm được; chưa biết trước mức cải thiện tối đa

Câu hỏi còn mở chung (tổng hợp của em từ cột cuối):

1. **Reward thưa.** Reward nhị phân không cho gradient khi không rollout nào thành công (SimpleVLA-RL, RIPT-VLA); reward quá trình (VLA-RL) lại dựa trên heuristic.
2. **Hiệu quả mẫu và reset.** RL online cần nhiều rollout và reset về cùng cảnh ban đầu; trên robot thật phải cần người ($\pi^*_{0.6}$).
3. **Log-likelihood cho policy flow.** Chưa có cách tốt nhất cho mọi model (π_{RL}: Flow-Noise hay Flow-SDE tuỳ model); RECAP và FPO tránh log-prob; DSRL tránh cả gradient qua base policy.

2. Diffusion và flow matching trong sinh hành động

Bảng 2: Ma trận so sánh diffusion (Diffusion Policy, DDPM) và flow matching (π_0, SmolVLA).

Tiêu chí	Diffusion (DDPM) — Diffusion Policy, arXiv:2303.04137	Flow matching — π_0 arXiv:2410.24164; SmolVLA arXiv:2506.01844
Điểm chung	Một mạng nhận *chunk action đang bị nhiễu* cùng bước thời gian/mức nhiễu, có điều kiện là quan sát, và trả ra một đại lượng (nhiễu hoặc vận tốc) để khử nhiễu dần từ Gauss về action.	
Kiến trúc	Điều kiện là quan sát O_t. Hai backbone: CNN 1D với FiLM theo O_t và bước k; hoặc transformer, token action nhiễu đi qua cross-attention với embedding quan sát.	VLM xử lý ảnh, câu lệnh và state (prefix). *Action expert* là transformer nhỏ xử lý token action và attend vào đặc trưng VLM; KV của prefix được cache nên mỗi bước tích phân chỉ tính lại phần action (π_0). SmolVLA (probe): VLM SmolVLM2-500M (16 lớp ngôn ngữ, vision 12 lớp); tổng 450,0M tham số, action expert 98,2M (16 lớp, rộng 0,75 lần VLM); prefix 177 token (2 camera × 64 + 48 ngôn ngữ + state); output chunk 50×7; expert cross-attention vào VLM, xen self-attention mỗi 2 lớp.
Mục tiêu huấn luyện	Lấy action sạch x^0, chọn bước k, cộng nhiễu ε^k; mạng *dự đoán nhiễu*: $\mathcal{L} = \text{MSE}(\varepsilon^k,\ \varepsilon_\theta(O_t,\ x^0 + \varepsilon^k,\ k))$.	Nội suy thẳng giữa action và nhiễu; theo quy ước code SmolVLA $x_t = t\varepsilon + (1-t)A$, $t=1$ là nhiễu thuần (probe; bài π_0/SmolVLA viết τ ngược chiều: $A^\tau = \tau A + (1-\tau)\varepsilon$). Mạng *dự đoán vận tốc*: $\|v_\theta(x_t, o) - (\varepsilon - A)\|^2$ (SmolVLA). t lấy theo phân phối Beta, ưu tiên mức nhiễu cao (π_0: Beta$((s-\tau)/s;\ 1{,}5,\ 1)$, $s = 0{,}999$).
Inference	Từ nhiễu Gauss x^K, lặp K bước $x^{k-1} = \alpha(x^k - \gamma\,\varepsilon_\theta(x^k, k) + \mathcal{N}(0, \sigma^2 I))$; α, γ, σ theo noise schedule. Train 100 bước; inference DDIM 10 bước, 0,1 s trên RTX 3080.	Lấy nhiễu một lần, tích phân ODE bằng Euler từ $t=1$ về 0: $x \leftarrow x - \Delta t\, v_\theta$. π_0 và SmolVLA dùng 10 bước (π_0, $\delta = 0{,}1$; probe `num_steps = 10`). SmolVLA: 96,5 ms/chunk (batch 1, RTX 5070 Ti; probe).
Đường từ nhiễu đến action	theo noise schedule α, γ, σ	đường thẳng (nội suy tuyến tính)
Tính ngẫu nhiên khi sinh	ở mọi bước (DDPM cộng nhiễu mỗi bước)	chỉ ở nhiễu ban đầu; phần còn lại tất định (ODE)
Log-likelihood của action	mỗi bước DDPM là một chuyển Gauss, nên log-prob của cả chuỗi khử nhiễu tính được	ODE tất định: log-likelihood chính xác phải tích phân divergence của v_θ dọc đường đi (như continuous normalizing flow), rất tốn; loss flow chỉ là cận dưới ($\pi^*_{0.6}$)
Hệ quả cho RL	policy gradient (PPO, GRPO) dùng trực tiếp log-prob theo từng bước khử nhiễu	cần một trong các hướng: (1) chèn nhiễu học được vào mỗi bước Euler, mỗi bước thành Gauss (π_{RL} Flow-Noise); (2) đổi ODE thành SDE giữ nguyên phân phối biên, thành MDP hai tầng (π_{RL} Flow-SDE); (3) bỏ log-prob, điều kiện policy theo advantage (RECAP); (4) xấp xỉ tỉ lệ likelihood bằng chênh lệch loss flow matching (FPO). SmolVLA cùng cấu trúc với π_0 nên (1), (2) áp dụng được bằng cách thêm nhiễu vào vòng Euler và thêm critic.

3. Tài liệu nền

- **Reward shaping** Thêm vào reward một số hạng $F = \gamma\Phi(s') - \Phi(s)$ (shaping theo thế năng) thì policy tối ưu không đổi. Đây là cơ sở cho thành phần tiến độ trong reward của phần Improve.
- **LIBERO** 130 task trong bốn bộ: Spatial, Object, Goal (mỗi bộ 10 task) và LIBERO-100 (LIBERO-90 + LIBERO-10/Long); mỗi task 50 demo do người điều khiển. Báo cáo dùng LIBERO-10 (Long).

Tài liệu tham khảo

- SimpleVLA-RL (2025). arXiv:2509.09674.

- π_{RL}: Online RL fine-tuning for flow-based VLA. arXiv:2510.25889.
- $\pi^*_{0.6}$ / RECAP. arXiv:2511.14759.
- RIPT-VLA (2025). arXiv:2505.17016.
- VLA-RL (2025). arXiv:2505.18719.
- Diffusion Policy. arXiv:2303.04137.
- π_0. arXiv:2410.24164.
- SmolVLA. arXiv:2506.01844.
- Flow Matching for Generative Modeling. arXiv:2210.02747.
- Flow Matching Policy Gradients (FPO). arXiv:2507.21053.
- Reinforcement Fine-Tuning of Flow-Matching Policies for VLA Models (π_0-FPO). arXiv:2510.09976.
- Steering diffusion policy with latent space RL (DSRL). arXiv:2506.15799.
- Policy invariance under reward transformations: theory and application to reward shaping. ICML.
- LIBERO. NeurIPS Datasets and Benchmarks. arXiv:2306.03310.